HW4

Greyson Knippel
CPSC-321- Fall 2026

October 15, 2026

Question 1

`border_full` uses `UNION` to combine the stored rows of `border` with a flipped copy (`country_code_2`, `country_code_1`), so every border appears in both directions. the 3 stored borders become 6 rows.

output

```
 country_code_1 | country_code_2 | border_length
----------------+----------------+---------------
 US             | CA             |          8891
 CA             | US             |          8891
 US             | MX             |          3145
 CA             | MX             |           500
 MX             | US             |          3145
 MX             | CA             |           500
(6 rows)
```

Question 2

threshold: border length > 400 (same as hw3 query 8).

output (hw4, using border_full)

```
 country_code |       country_name
--------------+--------------------------
 CA           | Canada
 US           | United States of America
(2 rows)
```

output (hw3 query 8, same starting data)

```
 country_code |       country_name
--------------+--------------------------
 CA           | Canada
 US           | United States of America
(2 rows)
```

comparison

both queries return the same rows: US and CA. the US has a higher gdp (46900 > 15300) and lower inflation (3.8 < 4.7) than MX, and it borders with length 3145 > 400. CA has a higher gdp (43100 > 15300) and lower inflation (2.4 < 4.7) than MX, which it borders with length 500 > 400. the hw3 query needed `OR` to check both columns of `border`, but `border_full` already has both directions, so the hw4 query only checks `country_code_1`.

Question 3

output

```
 country_code | country_name
--------------+--------------
 DE           | Germany
 IS           | Iceland
(2 rows)
```

examples

relevant stored rows:

```
country: ('DE', 'Germany', 52100, 2.2)
         ('US', 'United States of America', 46900, 3.8)
border:  ('MX', 'US', 3145)
         ('US', 'CA', 8891)
         ('CA', 'MX', 500)
```

matching: DE has no rows in `border`, so it never appears as `country_code_1` in `border_full`. it is not removed by the `EXCEPT` and shows up in the result.
nonmatching: US is in the stored row `('MX', 'US', 3145)`, so `border_full` has a row with `country_code_1 = 'US'`. the second half of the `EXCEPT` returns US, so it is removed.

Question 4

output

```
 country_code | country_name
--------------+--------------
 DE           | Germany
 IS           | Iceland
(2 rows)
```

comparison

the output matches question 3. the `LEFT JOIN` keeps every country, and countries with no border get `NULL` in the `border_full` columns, so `IS NULL` keeps only the countries with no borders.

Question 5

output

```
 country_code | country_name |  gdp
--------------+--------------+-------
 DE           | Germany      | 52100
 IS           | Iceland      | 52100
(2 rows)
```

examples

relevant stored rows:

```
country: ('DE', 'Germany', 52100, 2.2)
         ('IS', 'Iceland', 52100, 3.1)
         ('US', 'United States of America', 46900, 3.8)
```

matching: DE and IS both have gdp 52100. no country has a higher gdp, so the join `c1.gdp < c2.gdp` never matches them, they are not removed by the `EXCEPT`, and both are returned (a tie).
nonmatching: US has gdp 46900, which is lower than DE's 52100, so the join matches US and it is removed by the `EXCEPT`.

Question 6

parameters: first execution is `('DE', 563311)`, second is `('US', 200000)`. for the first execution, dresden has population exactly 563311 (equal to the minimum, so it is included) and nuremberg has 523026 (below the minimum, so it is excluded).

output (first execution)

```
 city_name | province_name | population
-----------+---------------+------------
 Munich    | Bavaria       |    1512491
 Dresden   | Saxony        |     563311
 Leipzig   | Saxony        |     616093
(3 rows)
```

output (second execution)

```
 city_name | province_name | population
-----------+---------------+------------
 Seattle   | Washington    |     737015
 Spokane   | Washington    |     228989
 Portland  | Oregon        |     652503
(3 rows)
```

Question 7

parameters: `large_cities('DE', 563311)`, the same as the first execution in question 6.

output

```
 city_name | population | province_name | area
-----------+------------+---------------+-------
 Munich    |    1512491 | Bavaria       | 70550
 Leipzig   |     616093 | Saxony        | 18450
 Dresden   |     563311 | Saxony        | 18450
(3 rows)
```

comparison

the city names, province names, and populations (munich, leipzig, dresden) match the first execution in question 6. this query also adds the province area and sorts by population from largest to smallest.

Question 8

parameters: `CALL add_province('CA', 'Quebec', 1542056, 'Montreal', 1762949);`

output

```
 province_name | country_code |  area   | city_name | population
---------------+--------------+---------+-----------+------------
 Quebec        | CA           | 1542056 | Montreal  |    1762949
(1 row)
```

Question 9

rentals of g rated films whose copy is from store 1 (`inventory.store_id`) and that have not been returned (`return_date IS NULL`), sorted by last name then first name.

output

```
 first_name | last_name |              email                |        title          |     rental_date
------------+-----------+-----------------------------------+-----------------------+---------------------
 DERRICK    | BOURQUE   | DERRICK.BOURQUE@sakilacustomer.org | MOSQUITO ARMAGEDDON   | 2006-02-14 15:16:03
 GARY       | COY       | GARY.COY@sakilacustomer.org       | CHITTY LOCK           | 2006-02-14 15:16:03
 EMILY      | DIAZ      | EMILY.DIAZ@sakilacustomer.org     | CONNECTION MICROCOSMOS | 2006-02-14 15:16:03
 FREDDIE    | DUGGAN    | FREDDIE.DUGGAN@sakilacustomer.org | GHOST GROUNDHOG       | 2006-02-14 15:16:03
 BETH       | FRANKLIN  | BETH.FRANKLIN@sakilacustomer.org  | BAKED CLEOPATRA       | 2006-02-14 15:16:03
 KENNETH    | GOODEN    | KENNETH.GOODEN@sakilacustomer.org | OPERATION OPERATION   | 2006-02-14 15:16:03
 JUDY       | GRAY      | JUDY.GRAY@sakilacustomer.org      | SUMMER SCARFACE       | 2006-02-14 15:16:03
 GAIL       | KNIGHT    | GAIL.KNIGHT@sakilacustomer.org    | HYDE DOCTOR           | 2006-02-14 15:16:03
 ALICIA     | MILLS     | ALICIA.MILLS@sakilacustomer.org   | SLEEPLESS MONSOON     | 2006-02-14 15:16:03
 HEATHER    | MORRIS    | HEATHER.MORRIS@sakilacustomer.org | LAWLESS VISION        | 2006-02-14 15:16:03
 ELMER      | NOE       | ELMER.NOE@sakilacustomer.org      | MINORITY KISS         | 2006-02-14 15:16:03
 JAY        | ROBB      | JAY.ROBB@sakilacustomer.org       | INTRIGUE WORST        | 2006-02-14 15:16:03
 DIANNE     | SHELTON   | DIANNE.SHELTON@sakilacustomer.org | CAT CONEHEADS         | 2006-02-14 15:16:03
 ALLISON    | STANLEY   | ALLISON.STANLEY@sakilacustomer.org | GRAPES FURY           | 2006-02-14 15:16:03
 LARRY      | THRASHER  | LARRY.THRASHER@sakilacustomer.org | PULP BEVERLY          | 2006-02-14 15:16:03
 MARGIE     | WADE      | MARGIE.WADE@sakilacustomer.org    | LUST LOCK             | 2006-02-14 15:16:03
 FRED       | WHEAT     | FRED.WHEAT@sakilacustomer.org     | SWEETHEARTS SUSPECTS  | 2006-02-14 15:16:03
 TYLER      | WREN      | TYLER.WREN@sakilacustomer.org     | BLANKET BEVERLY       | 2006-02-14 15:16:03
(18 rows)
```

Question 10

customer 43's rentals where `return_date > rental_date + INTERVAL '7 days'` or `return_date IS NULL`, sorted by rental date.

output

```
        title        |     rental_date     |     return_date
---------------------+---------------------+---------------------
 PATIENT SISTER      | 2005-07-06 09:25:56 | 2005-07-14 12:10:56
 MURDER ANTITRUST    | 2005-07-09 09:00:11 | 2005-07-17 07:26:11
 METAL ARMAGEDDON    | 2005-08-17 09:11:52 | 2005-08-25 05:03:52
 JET NEIGHBORS       | 2005-08-22 01:54:52 | 2005-08-29 02:24:52
 POLLOCK DELIVERANCE | 2006-02-14 15:16:03 |
 MOTIONS DETAILS     | 2006-02-14 15:16:03 |
(6 rows)
```

JET NEIGHBORS was returned 7 days and 30 minutes after it was rented, so it just qualifies. the last two rows have no return date, so they qualify through `IS NULL`.

Question 11

G-rated films with no copies in the inventory of either store, using a `LEFT JOIN` from `film` to `inventory` and keeping rows where `inventory_id IS NULL`.

output

```
 film_id |       title
---------+-------------------
     128 | CATCH AMISTAD
     217 | DAZED PUNK
     318 | FIREHOUSE VIETNAM
     419 | HOCUS FRIDA
     497 | KILL BROTHERHOOD
     860 | SUICIDES SILENCE
     954 | WAKE JAWS
(7 rows)
```

Question 12

actors in a horror film at least 170 minutes long, `INTERSECT` actors in a sci fi film at least 170 minutes long. `INTERSECT` removes duplicates, so each actor appears once.

output

```
 actor_id | first_name |  last_name
----------+------------+--------------
      158 | VIVIEN     | BASINGER
       60 | HENRY      | BERRY
      179 | ED         | GUINESS
        5 | JOHNNY     | LOLLOBRIGIDA
```

```
      195 | JAYNE      | SILVERSTONE
       72 | SEAN       | WILLIAMS
       68 | RIP        | WINSLET
(7 rows)
```